

Assignment 3 - Bridge Pattern

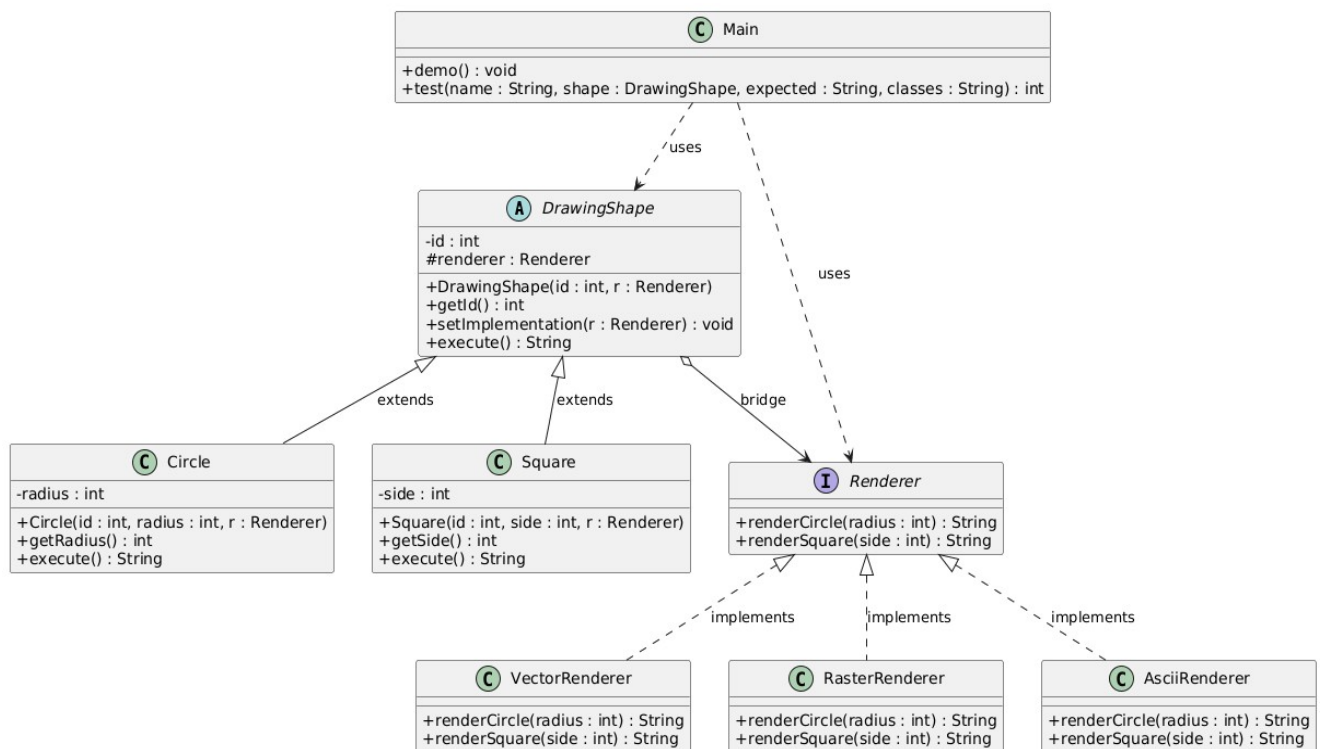
Alish Medina, SE-2538

1. Two Independent Dimensions and Why Bridge Fits

The drawing application varies along two separate axes.

1. The first axis is the abstraction: what shape is being drawn. Circle (A1) and Square (A2) extend the abstract base DrawingShape. Each carries its own domain data - radius for Circle, side for Square - and its own ID.
2. The second axis is the implementation: how a shape is drawn. VectorRenderer (I1), RasterRenderer (I2), and AsciiRenderer (I3) all implement the Renderer interface, each producing a different text rendering.
3. These two axes change for unrelated reasons. A new shape is added because the application needs a new geometry. A new renderer is added because a new output format is required. Neither change should force edits in the other hierarchy.
4. Bridge fits because it lets exactly this happen. Instead of creating a subclass for every shape-renderer pair - VectorCircle, RasterCircle, AsciiCircle, VectorSquare, and so on - DrawingShape holds a single interface-typed reference renderer and delegates rendering to it. Adding AsciiRenderer in this project required writing one new class and updating only Main. The abstraction hierarchy was not touched.

2. UML Class Diagram



The hollow diamond on `DrawingShape` is the bridge: an interface-typed reference held by composition. `Main` depends on both hierarchies: it constructs shapes and renderers and wires them together.

3. Five Annotated Code Excerpts

3.1 Separation of responsibilities - DrawingShape

```
public DrawingShape(int id, Renderer renderer) { 3 usages  Medina *
    this.id = id;
    this.renderer = renderer;
}
```

DrawingShape knows nothing about Vector, Raster, or ASCII. It knows it has a renderer and can delegate to it. The "how" of drawing lives entirely on the other side of the bridge. Benefit: a change in rendering logic cannot ripple into the shape hierarchy.

3.2 Meaningful names - Renderer interface

```
public interface Renderer { 12 usages 3 implementations  Medina *
    String renderCircle(int radius); 1 usage 3 implementations  Medina
    String renderSquare(int side); 1 usage 3 implementations  Medina
}
```

Each method name states exactly what it does and takes only the data it needs — no shape object passed in. Benefit: a reader understands the entire contract of the interface from two lines, and implementers have no ambiguity about what to return.

3.3 Small focused methods - Circle.execute()

```
@Override 3 usages  Medina
public String execute() {
    return renderer.renderCircle(radius);
}
```

One line, one responsibility: forward the radius to whichever renderer is attached. No branching, no formatting, no state mutation. Benefit: the delegation point is trivially testable and easy to change.

3.4 Avoidance of duplication - Main.test

```
public static int test(String name, DrawingShape shape, String expected, String classes) {
    String result = shape.execute();
    if (result.equals(expected)) {
        System.out.println(name + " PASS | " + classes + " | result=" + result);
        return 1;
    }
    System.out.println(name + " FAIL | " + classes
        + " | result=" + result
        + " | expected=" + expected);
    return 0;
}
```

Six of the seven checks use the same compare-and-print workflow. Writing it once means a change to the output format touches exactly one method. Benefit: consistency across all checks and no copy-paste drift.

3.5 Encapsulation of state - Circle

```

public class Circle extends DrawingShape { 14 usages  Medina *
    private final int radius; 3 usages

    public Circle(int id, int radius, Renderer renderer) { 9
        super(id, renderer);
        this.radius = radius;
    }

    public int getRadius() { 2 usages new *
        return radius;
    }
}

```

Radius is private final. It is set once at construction and never mutated. Benefit: object identity and domain data are stable, which is exactly what T5 proves — the same object, same ID, same radius, only the renderer changes.

4. Adding I3, What Stayed Unchanged, Trade-off, and Bridge vs. Adapter

How I3 was added. AsciiRenderer was written as a new class implementing Renderer. It contains two methods, renderCircle and renderSquare, returning strings prefixed with ASCII. Main was updated to instantiate it and add checks T6 and T7. Nothing else was touched.

What stayed unchanged. The diff from base commit 9a7249d to submitted commit e7728c9 contains exactly two files under src/: the new AsciiRenderer.jav and the modified Main.java. DrawingShape, Circle, Square, Renderer, VectorRenderer, and `RasterRenderer` are identical between the two commits. This is the concrete evidence that the bridge allows a new implementation to be plugged in without any edit to the abstraction side.

One trade-off of Bridge. Bridge introduces indirection. Instead of one class calling a rendering method directly, the program has two hierarchies plus an interface between them. For a program with a single shape and a single renderer, this structure would be over-engineering. It pays off here because both axes genuinely vary and the assignment requires swapping renderers on the same object at runtime.

How Bridge differs from Adapter. Adapter is applied after the fact to make an existing incompatible interface fit a new client. It reshapes an interface but does not change the class structure or enable independent variation. Bridge is applied up front to factor one whole dimension of variation out behind an interface, so both sides can grow independently. In this project there is no legacy class to adapt - both shape classes were written knowing about the Renderer interface. The intent is extension without combinatorial classes, not compatibility with an existing interface.

5. Build and Run Commands

```
javac --release 17 -encoding UTF-8 -d out "sources.txt"
```

```
java -cp out Main --demo
```

6. Expected Results (T1–T7)

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | Circle + VectorRenderer -> RasterRenderer | sameObject=true | sameId=true | stateUnchanged=true | before=VECTOR circle radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS
```

7. Conclusion

Bridge let the shape hierarchy and the renderer hierarchy vary independently, connected only by an interface-typed reference in `DrawingShape`. All seven checks passed: T1–T4 covered every original shape–renderer combination, T5 proved that one `Circle` object can switch renderers at runtime while keeping its identity, ID, and radius, and T6–T7 confirmed that the new `AsciiRenderer` plugged in without touching any existing class. The extension diff contains only `AsciiRenderer.java` and `Main.java`, which is the clearest evidence that the pattern worked as intended. The cost is indirection — overkill for one shape and one renderer, but worth it when both dimensions genuinely vary. Unlike `Adapter`, which retrofits an old interface to a new client, `Bridge` is a structural decision made up front so both sides can grow without affecting each other.